Algorithms for Arithmetic

Easier (linear) to multiply with 10^a and additions are linear time.

But is the procedure more efficient??

Time for multiplying 2 n digit numbers

$$T(n) = 4T(n/2) + O(n)$$

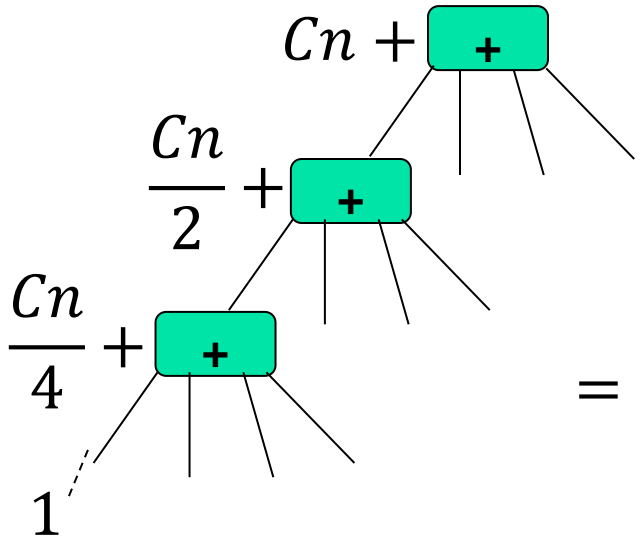
Linear in n (Cn)

$$T(n) = 1, n = 1$$

$n \rightarrow$ is a perfect power of 2 or we make it so!

This will make the number of digit increase at max 2 times of the actual!

$$n \rightarrow 2n$$



Recursion tree:

$$Cn + \frac{4Cn}{2} + 4 \times \frac{4Cn}{4} + 4^2 \times \frac{4Cn}{8} + \cdots$$

$$= Cn + 2Cn + 4Cn + 8Cn + \cdots + 2^k Cn$$

$$2^k = n, k = \log_2 n$$

$$= Cn(1 + 2 + 2^2 + 2^3 + \cdots + 2^k)$$ Finite sum series!

$$= Cn\left(\frac{2^{k+1} - 1}{2 - 1}\right) = Cn(2n - 1) \rightarrow O(n^2)$$

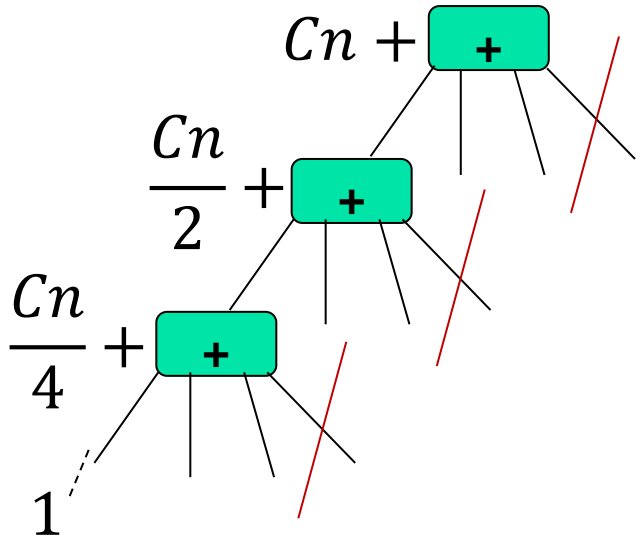
So equally complex!!! In fact it will probably be slower!

Gauss to rescue more than a century later in 1960s…

$$X_h Y_h 10^n + (X_h Y_l + X_l Y_h)10^{\frac{n}{2}} + X_l Y_l$$

$$(X_h + X_l)(Y_h + Y_l) - X_h Y_h - X_l Y_l$$

One multiplication, NOT two



$$Cn + \frac{3Cn}{2} + 3 \times \frac{3Cn}{4} + 3^2 \times \frac{3Cn}{8} + \cdots$$

$$= Cn(1 + 3/2 + (3/2)^2 + \cdots + (3/2)^k) \quad = 2Cn((3/2)^{(\log_2 n)+1} - 1)$$

$$= 2Cn(3/2)^{(\log_2 n)+1} - 2Cn$$

$$= \frac{3}{2}2Cn\left(\frac{3}{2}\right)^{\log_2 n} - 2Cn = 3Cn\left(\frac{3^{\log_2 n}}{n}\right) - 2Cn$$

$$3C3^{\log_2 n} - 2Cn = 3C\left(2^{\log_2 3}\right)^{\log_2 n} - 2Cn$$

$$= 3Cn^{\log_2 3} - 2Cn \leq 3Cn^{\log_2 3} \quad O(n^{log_2 3})$$

$\log_2 3 \rightarrow 1.585$

Karatsuba's algorithm!

Invented in 1960s

It has higher constant /linear comp factor… so don't use when *n* is very small!

Tremendous progress since then:

Dividing into more parts? $O(n^{log_3 5}) \longleftrightarrow O(n^{log_p(2p-1)})$

Further… $O(nlog(n)log(\log n))$ considering a number as a polynomial of 10^

In 2019: $O(nlog(n))$ Best procedure till now!

https://www.jstor.org/stable/10.4007/annals.2021.193.2.4

Karatsuba's algorithm:

A divide-and-conquer algorithm for integer multiplication.

```
function multiply(x, y)
Input:  Positive integers x and y, in binary
Output:  Their product

n = max(size of x, size of y)
if n = 1:  return xy

x_L, x_R = leftmost ⌈n/2⌉, rightmost ⌊n/2⌋ bits of x
y_L, y_R = leftmost ⌈n/2⌉, rightmost ⌊n/2⌋ bits of y

P_1 = multiply(x_L, y_L)
P_2 = multiply(x_R, y_R)
P_3 = multiply(x_L + x_R, y_L + y_R)
return P_1 × 2^n + (P_3 − P_1 − P_2) × 2^(n/2) + P_2
```

Fibonacci Numbers

On Fibonacci Sequence:

$$0, 1, 0+1=1, 1+1=2, 1+2=3, 2+3=5, 3+5=8 \ \ldots$$

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \ldots,$$

Fibonacci numbers F_n → Fibonacci index

Mathematically:

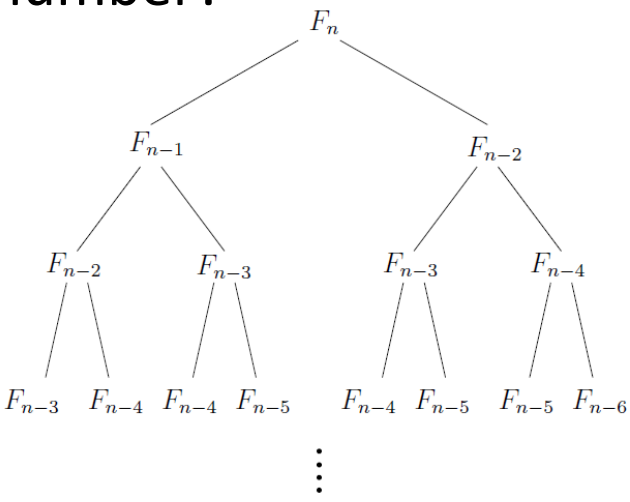
$$F_n = \begin{cases} F_{n-1} + F_{n-2} & \text{if } n > 1 \\ 1 & \text{if } n = 1 \\ 0 & \text{if } n = 0\,. \end{cases}$$

Algorithm to generate the n^{th} Fibonacci number?

***Algorithm 1* – Recursion**

```
function fib1(n)
if n = 0:   return 0
if n = 1:   return 1
return fib1(n − 1) + fib1(n − 2)
```

} return n



Efficiency:

finite time (basic or more)

$$T(n) = \begin{cases} 0 + t, if\ n \leq 1 \\ T(n-1) + T(n-2) + 1, if\ n > 1 \end{cases}$$

Fibonacci number increases similar to exponentials of *2*!

$$F_n = F_{n-1} + F_{n-2} \geq 2F_{n-2} \geq 4F_{n-4} \geq 8F_{n-6} \geq \cdots \geq 2^{\left\lfloor \frac{n-1}{2} \right\rfloor}$$

$F_2\ or\ F_1$

$\left\lfloor \frac{n-1}{2} \right\rfloor$

Number of binary digits!

$(F_{n-1} \geq F_{n-2})$

How about $\leq$?

So does its efficiency! $T(n-1) \geq T(n-2)$

$$T(n) = T(n-1) + T(n-2) + 1 \geq 2^{\frac{n}{2}}t + 2^{\frac{n}{2}-1}$$

at least exponential!

***Algorithm 2* – Iteration**

```
function fib2(n)
if n = 0 return 0
create an array f[0...n]
f[0] = 0, f[1] = 1
for i = 2...n:
    f[i] = f[i-1] + f[i-2]
return f[n]
```

n=1 also

$A(init\ f[0])$, $B(init\ f[1]), tmp$ are sufficient

A simple example of dynamic programming!

Efficiency:

Inner loop (for) runs $n-1$ times!

So $T(n) = C_1(n-1) + C_2$ Linear in n!

Exponentially superior!

* If we consider the complexity of the addition within loop, with the number of digits being *N* for f_n :

`fib2` $\longrightarrow$ $Nn \gtrsim n^2$ *N* digit proportional to *n*

`fib1` $\longrightarrow$ $\sim n f_n$?

***Algorithm 3* – Fast matrix powering**

$$\underbrace{\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}}_{A} \begin{bmatrix} F_1 \\ F_0 \end{bmatrix} = \begin{bmatrix} F_1 + F_0 \\ F_1 \end{bmatrix} = \begin{bmatrix} F_2 \\ F_1 \end{bmatrix}$$

$$\Longrightarrow \quad A^n \begin{bmatrix} F_1 \\ F_0 \end{bmatrix} = \begin{bmatrix} F_{n+1} \\ F_n \end{bmatrix}$$

Efficiency:

$A^n \longrightarrow Mn$

Number of operations to multiply a pair of 2x2 and 2x2 matrices

So $T(n) = Mn + L \longrightarrow$ Number of operations to multiply a 2x2 matrix with a 2x1 matrix

Linear in n!

$A^{n-1} \rightarrow A^n$

$$\begin{bmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{bmatrix} \rightarrow \begin{bmatrix} F_n + F_{n-1} & F_{n-1} + F_{n-2} \\ F_n & F_{n-1} \end{bmatrix}$$

Fast exponentiation by repeated squaring -

Consider: 9^{71} ↗ n

$$\lfloor log_2(n) \rfloor = \lfloor log_2(71) \rfloor = 6$$

$9^1, 9^1 \times 9^1 = 9^2, 9^2 \times 9^2 = 9^4, 9^4 \times 9^4 = 9^8, 9^8 \times 9^8 = 9^{16},$
$9^{16} \times 9^{16} = 9^{32}, 9^{32} \times 9^{32} = 9^{64} \longrightarrow 2^6$

Now: $71_{10} = 1000111_2$

$\log(n)$ complexity

$9^{71} = 9^{64} \times 9^4 \times 9^2 \times 9^1!$

Using less than $3log_2(n)$ multiplications and look up!

$$A^n = \prod_{i=0}^{\lfloor \log_2 n \rfloor} A^{b_i 2^i}$$

where $b_i s$ are binary digits of n

$$T(n) \leq Mlog_2(n) + L$$

Exponentially superior??!!

Algorithm 4 – via Eigendecomposition

$A^n v$

$\downarrow$

$A^n = Q\Lambda^n Q^{-1}$

$A = Q\Lambda Q^{-1}$

$\downarrow$

diagonal

$$A^2 = Q\Lambda Q^{-1} Q\Lambda Q^{-1} = Q\Lambda I\Lambda Q^{-1}$$

The eigenvalues in Λ do not change with n!

Our case:

$$F_n \leftarrow A^{n-1} v \qquad A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \qquad v = \begin{bmatrix} F_1 \\ F_0 \end{bmatrix}$$

One time constant cost decomposition:

$$Q = \begin{bmatrix} \phi & 1-\phi \\ 1 & 1 \end{bmatrix} \qquad \Lambda = \begin{bmatrix} \phi & 0 \\ 0 & 1-\phi \end{bmatrix} \qquad \phi = \frac{1+\sqrt{5}}{2}$$

Take $$\begin{bmatrix} F_1 \\ F_0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{5}} \left(\begin{bmatrix} \phi \\ 1 \end{bmatrix} - \begin{bmatrix} 1-\phi \\ 1 \end{bmatrix} \right)$$

$$A^{n-1}v = A^{n-1}\left(\frac{1}{\sqrt{5}}\left(\begin{bmatrix}\phi\\1\end{bmatrix} - \begin{bmatrix}1-\phi\\1\end{bmatrix}\right)\right)$$

$$A^{n-1}v = \frac{1}{\sqrt{5}}\left(A^{n-1}\begin{bmatrix}\phi\\1\end{bmatrix} - A^{n-1}\begin{bmatrix}1-\phi\\1\end{bmatrix}\right)$$

$$A^{n-1}v = \frac{1}{\sqrt{5}}\left(\phi^{n-1}\begin{bmatrix}\phi\\1\end{bmatrix} - (1-\phi)^{n-1}\begin{bmatrix}1-\phi\\1\end{bmatrix}\right)$$

$$A^{n-1}v = \frac{1}{\sqrt{5}}\begin{bmatrix}\phi^{n} - (1-\phi)^{n}\\ \phi^{n-1} - (1-\phi)^{n-1}\end{bmatrix}$$

$$f(n) = \frac{1}{\sqrt{5}}(\phi^{n} - (1-\phi)^{n})$$

Exact relation!

Spend $O(n)$ space and get constant time result?!

More…

Closed Form: $f(n) = \frac{1}{\sqrt{5}}(g^n - r^n)$

The golden ratio where F_n/F_{n-1} converges $\leftarrow$ $g = \frac{1+\sqrt{5}}{2}$ $r = \frac{1-\sqrt{5}}{2}$

Irrational numbers!

No memory.. Time complexity…

Multiplication of scalars!

$O(\log n)$

n only as Fibonacci index

Fibonacci numbers are integers….

What about error creeping in for higher n by representing irrational numbers as floating point?

Asymptotic Notations

Let f, g are functions mapping $\mathbb{Z}^+$ to $\mathbb{Z}^+$

"f grows no faster than g"

"Big oh" $f = O(g)$ $if\ \exists c > 0\ s.t.\ \forall n\ f(n) \leq cg(n)$

"little oh" $f = o(g)$ $if\ \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

"f grows slower than g"

"Big Omega" $f = \Omega(g)\ if\ g = O(f)$

"little Omega" $f = \omega(g)\ if\ g = o(f)$

"Theta" $f = \Theta(g)\ if\ f = O(g)\ \&\ f = \Omega(g)$

Rough Analogies:

O	o	Ω	ω	Θ
$\leq$	$<$	$\geq$	$>$	$=$

Big-O notation lets us focus on the big picture

1. Multiplicative constants can be omitted: $14n^2$ becomes n^2.
2. n^a dominates n^b if $a > b$: for instance, n^2 dominates n.
3. Any exponential dominates any polynomial: 3^n dominates n^5 (it even dominates 2^n).
4. Likewise, any polynomial dominates any logarithm: n dominates $(\log n)^3$. This also means, for example, that n^2 dominates $n \log n$.

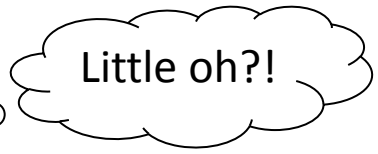
Examples:

- $f_1(n) = n^2 \qquad f_2(n) = 2n + 20$

$$\frac{f_2(n)}{f_1(n)} = \frac{2n+20}{n^2} \leq 22 \quad \text{for all } n$$

$$f_1(n)/f_2(n) = n^2/(2n+20)$$ Keeps growing

$$f_2 = O(f_1) \qquad f_1 \neq O(f_2)$$



- $f_3(n) = n + 1$

$$\frac{f_2(n)}{f_3(n)} = \frac{2n+20}{n+1} \leq 11 \qquad \frac{f_3(n)}{f_2(n)} = \frac{n+1}{2n+20} \leq \frac{1}{2}$$

$$f_2 = O(f_3) \qquad f_3 = O(f_2)$$

$$f_2 = \Theta(f_3), f_3 = \Theta(f_2)$$

- f(n)=$3n^3 + n^2 + \log n$ g(n)=n^3

$$\lim_{n\to\infty} \frac{3n^3 + n^2 + \log n}{n^3} = 3 \qquad f = O(n^3) \qquad f = \Theta(n^3)!$$

f can be for an algo

- f(n)=$\begin{cases} 2n^3, n \text{ is odd} \\ n^3, n \text{ is even} \end{cases}$ $\lim_{n\to\infty} \frac{2n^3}{n^3} = 2, \lim_{n\to\infty} \frac{n^3}{n^3} = 1$

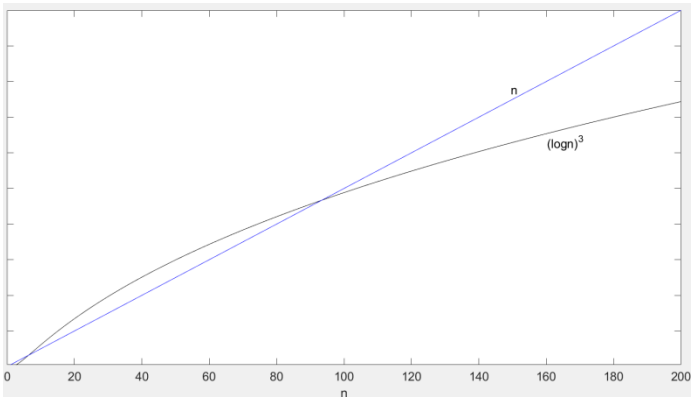
Limit keeps changing! But: $f = O(n^3)$

- f(n)=$\begin{cases} n^2, n \text{ is odd} \\ n, n \text{ is even} \end{cases}$

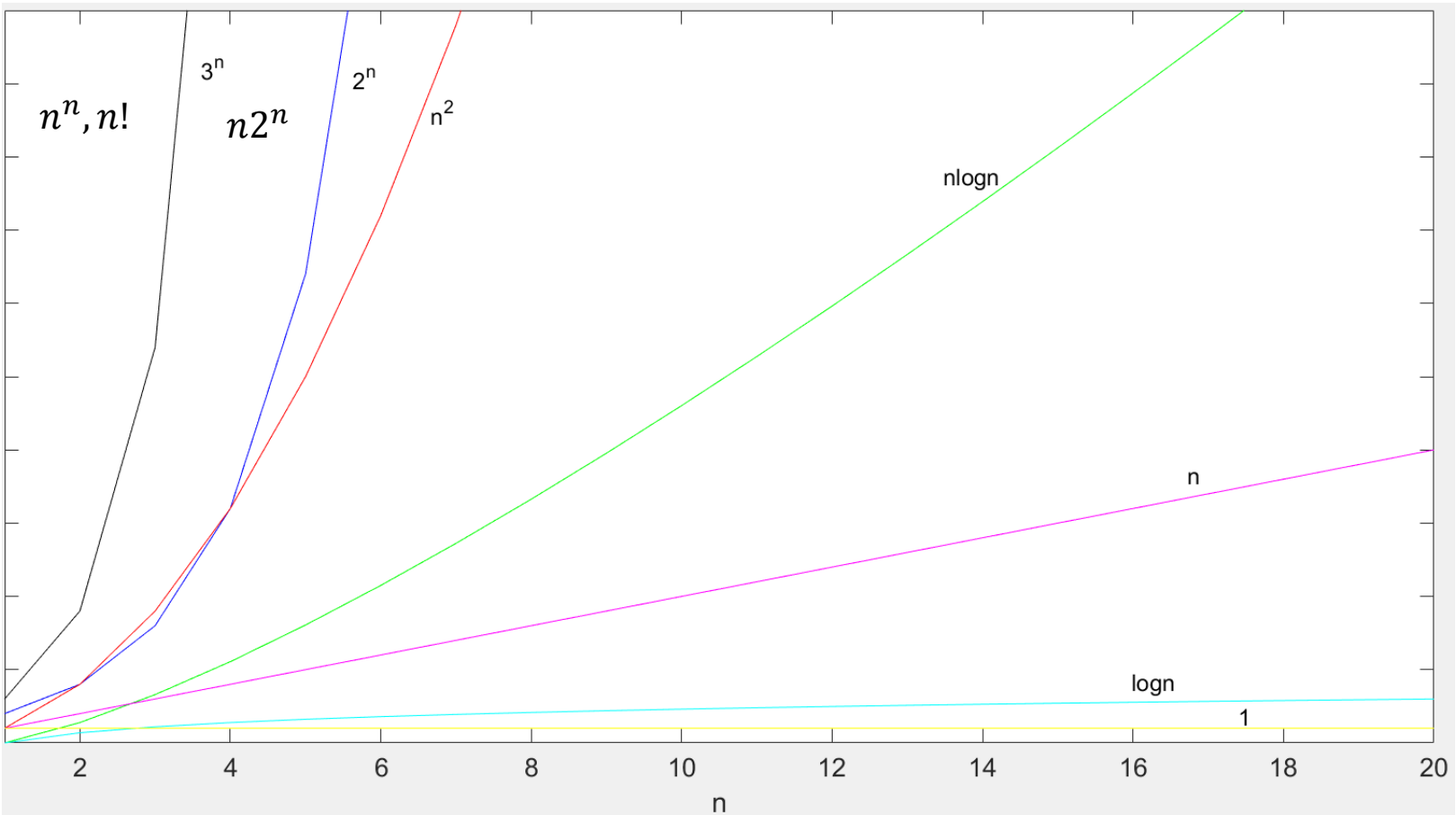
$$f = O(n^2)$$
$$f \neq o(n^2)$$
$$f \neq \Theta(n^2)$$

$g = n;\ g = o(f)?\ only\ odd\ n\ ;\ g = \Theta(f)?\ only\ even\ n.$

$So \Rightarrow g = O(f),\ f = \Omega(g) \rightarrow \boldsymbol{f = \Omega(n)}$



$n \geq 1$



Example complexity plots